

Hybrid Policy Optimization in Discontinuous MDPs: Combining Pathwise and Model-Free Gradient Estimators

Matias Alvo

1. Short summary

Our goal throughout this work is to develop a principled framework for reinforcement learning in environments with discontinuities, where standard pathwise gradient methods struggle. We introduce a modeling approach that explicitly captures the structure of such problems, enabling the use of pathwise gradients in regions where smoothness is available. Building on this framework, we propose methods that combine differentiable simulation with model-free gradient estimators, allowing us to leverage the strengths of both in a unified and theoretically grounded manner.

1.1. High-Level Modeling Approach

Starting point: an MDP. We consider a finite-horizon MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$ with known transition f , known per-period cost c , and observable exogenous noise $\xi_{1:T} \in \Xi$ (a “hindsight RL” setup (Alvo et al. 2023, Sinclair et al. 2022, Madeka et al. 2022, Xie et al. 2024)). For a policy π and initial state S_1 , we evaluate performance by simulating rollouts on an *offline* dataset of noise traces.

From MDP to piecewise-smooth structure. Many operations problems are *piecewise smooth*: the action set is a union of intervals and, within each interval, c and f are smooth, while *global* discontinuities may arise at interval boundaries (e.g., large fixed costs). We formalize this as a **piecewise-smooth MDP (PS-MDP)** in which: (i) $\mathcal{A} = \bigcup_{j=1}^J \mathcal{B}^j$ is a disjoint union of connected intervals, and (ii) for each j , the restrictions $c|_{\mathcal{B}^j}$ and $f|_{\mathcal{B}^j}$ are continuous and a.e. differentiable.

Factored representation that isolates smooth pieces. We represent the PS-MDP as a *factored* MDP with a discrete *mode* $x \in \{e_1, \dots, e_J\}$ choosing the interval and continuous *within-mode* action $b = (b^1, \dots, b^J)$ with $b^j \in \mathcal{B}^j$. Costs and transitions are then written as

$$c(s, (x, b), \xi) = \langle x, C(s, b, \xi) \rangle, \quad f(s, (x, b), \xi) = \langle x, F(s, b, \xi) \rangle,$$

so only the active coordinate j (where $x^j = 1$) affects dynamics and costs. This **factoring does not change the problem**: it is a representation that preserves values and optimal policies while *exposing* the within-piece smooth structure.¹

1.2. Gradient Estimation for parameterized policies

We compute policy gradients using the Stochastic Computation Graph (SCG) framework of Schulman et al. (2015a), which provides a flexible and intuitive method for deriving unbiased gradient estimators in models with both stochastic and deterministic components. We omit detail on this framework here for brevity.

Consider parameter spaces $\Phi = \mathbb{R}^{d_\phi}, \Theta = \mathbb{R}^{d_\theta}$, for some $d_\phi, d_\theta \in \mathbb{Z}$, and parameterized *policy components* $\tilde{\pi}_\phi^\mathcal{X}, \tilde{\pi}_\theta^\mathcal{B}$, for each $\phi \in \Phi$, and $\theta \in \Theta$, respectively. We will typically search over classes of neural policies, with ϕ and θ defining the weights of the neural networks. We say that a component $\tilde{\pi} \in \{\tilde{\pi}_\phi^\mathcal{X}, \tilde{\pi}_\theta^\mathcal{B}\}$ is *stochastic* if its output is a probability distribution over some set, and say it is *deterministic* if it maps to a fixed value. This resembles the concept of stochastic and deterministic policies. Policy components $\tilde{\pi}_\phi^\mathcal{X}$ and $\tilde{\pi}_\theta^\mathcal{B}$ can be stochastic or deterministic mappings from $s \in \mathcal{S}$ to the selector set $\mathcal{X}^J$ and $\mathcal{B}$, respectively.

Fully stochastic policies. When both components are stochastic, then we can use the standard score-function estimator to obtain unbiased gradients (see Equation (13)).

Hybrid policies. Now, consider the case in which $\tilde{\pi}_\phi^\mathcal{X}$ defines a stochastic distribution over discrete modes and $\tilde{\pi}_\theta^\mathcal{B}$ outputs deterministic values for each mode. Under mild regularity conditions, the gradient of the expected total cost can be expressed as

$$\mathbb{E}_{(s,a) \sim d_\infty^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}} \left[\nabla_{(\phi, \theta)} \log \tilde{\pi}_\phi(x | s) Q_t^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}(s, a) + (\mathbf{0}_{d_\phi}, \nabla_\theta c(s, a)) \right] = \nabla_{(\phi, \theta)} J(\phi, \theta). \quad (1)$$

In the previous equation, note that the score-function term considers only the policy component $\tilde{\pi}_\phi$, while the gradient is taken with respect to (ϕ, θ) . This term reflects the direct influence of ϕ on $\tilde{\pi}_\phi$ and, also, the influence of θ via its influence on s (via **all** deterministic paths connecting θ and s). The second term reflects the influence of θ in the costs (via **all** deterministic paths connecting θ and the cost), while representing that ϕ does not deterministically influence costs.

1.3. Agents we implemented

All agents operate on the *same factored environment* (same state, same cost/transition, same dataset of noise traces).

1. **PPO (fully stochastic on factored space).** Vanilla PPO (Schulman et al. 2017): sample a mode X_t and then sample B_t *conditional* on that mode; train with fully score-function gradients.

¹Formally, for every policy π in the original MDP, there is a factored-policy $\tilde{\pi} = (\tilde{\pi}^\mathcal{X}, \tilde{\pi}^\mathcal{B})$ with identical rollouts under the same noise realizations; hence $\min_\pi J(\pi) = \min_{\tilde{\pi}} J(\tilde{\pi})$.

2. **Hybrid-PPO.** We use the unbiased gradient estimator for hybrid policies: the discrete mode is updated with a PPO-style score-function objective, while the continuous component is trained via standard SGD on the pathwise and score-function terms that depend on it.
3. **ST-HDPO (ours).** We use the straight-through trick on the discrete action, and use HDPO with the fully-pathwise gradient estimator.

1.4. Results

We evaluated our agent in two settings, both involving a single store facing per-unit underage and holding costs, as well as a large fixed cost. The first setting considers a backlogged demand assumption and Poisson demand with mean 10. The second setting considers realistic demand traces from the Favorita dataset and a lost demand assumption. Tables 1 and 2 show the loss obtained in the best run for each fixed cost value in each setting. On the first setting we report cost, while in the second we report -profit (so in both cases lower is better).

For setting 1, we observe that PPO and Hybrid-PPO perform similarly, and both obtain near-optimal results. ST-HDPO performs significantly worse, with gaps typically of around 2%. On the setting with realistic demand, we observe that Hybrid-PPO outperforms ST-HDPO by a significant margin of around 0.5 – 1.0%, and outperforms PPO by margins of around 3 – 4%. This reveals that PPO can typically work well with synthetic demand, but has a hard time when facing realistic demand (**Note: I tried a plethora of tricks for each agent, and chose the best configuration for each. I also considered a version of Hybrid-PPO in which I disable the score-function term for the parameters of the continuous action and, surprisingly, it works equally well. I have to look into this further.**)

Table 1 Validation loss (cost per period) and optimality gaps (%) under synthetic demand.

Agent	Fixed Cost = 30	Fixed Cost = 40	Fixed Cost = 50	Fixed Cost = 64
PPO	27.30 (+0.55%)	30.43 (+0.03%)	33.36 (0.00%)	37.18 (+0.32%)
Hybrid-PPO	27.13 (−0.07%)	30.44 (+0.07%)	33.39 (+0.09%)	37.11 (+0.13%)
ST-HDPO	27.79 (+2.36%)	31.11 (+2.27%)	34.09 (+2.19%)	37.64 (+1.57%)

Table 2 Validation loss (- profit) under realistic demand.

Agent	Fixed Cost = 30	Fixed Cost = 40	Fixed Cost = 50	Fixed Cost = 64
PPO	-351.48	-345.19	-338.75	-333.52
Hybrid-PPO	-361.49	-355.95	-350.56	-343.75
ST-HDPO	-360.38	-353.92	-348.15	-339.83

1.5. Next steps

These are possible next steps

- Consider a high-dimensional continuous action for each discrete mode. An example of this is a setting where many products share a truck, so that the discrete action is whether to “send the truck” and the vector continuous action specifies order quantities per product. I believe this will further break PPO relative to the other 2, since score-function gradients should see its variance increased, whereas pathwise gradients should not suffer as much.
- Other sources of discontinuities, possibly in the transition function. I already finished coding a setting in which the lead time depends on the number of units ordered as a step-function. I believe this setting is inherently harder, since changes in the discrete action (*i.e.*, which lead time mode to choose) propagate to future states.
- Some setting where there are many discrete modes. One example of this is a setting where there is a fixed cost that increases after hitting various thresholds. I believe that as we increase the number of modes, the ST-HDPO gradient should benefit, since it is the only agent that considers pathwise gradients for the discrete actions.

2. Introduction

Differentiable simulation has emerged as a powerful paradigm for solving sequential decision-making problems. This technique has shown notable success in the robotics domain (Hu et al. 2019, Levine et al. 2016), leading to its recent adoption in operations problems such as inventory management (Alvo et al. 2023, Madeka et al. 2022) and queueing control (Che et al. 2024). The success of differentiable simulation is underpinned by its use of pathwise gradients, which typically exhibit lower variance than score-function (also known as REINFORCE) gradient estimators (Mohamed et al. 2020), which are ubiquitous in the Reinforcement Learning (RL) literature. However, the conditions required for the unbiasedness of pathwise gradient estimators are more stringent than those for their score-function counterparts. For example, discontinuities in either the cost or transition functions cause pathwise gradients to become biased. Moreover, if the problem involves discrete actions, a pathwise gradient is typically “uninformative”.

While pathwise methods such as HDPO (Alvo et al. 2023) offer low-variance gradient estimates in smooth systems, they are fundamentally incompatible with environments involving large discontinuities. Using smooth surrogates for discontinuous transitions or cost functions can lead to misleading gradients—particularly near thresholds or fixed-cost regions. For instance, in an inventory control problem with large fixed costs, a steep approximation near zero may produce excessively large gradients that dissuade the agent from increasing its action, even when doing so would reduce overall cost. In contrast, score-function estimators offer unbiased updates in such

cases, albeit with high variance. This raises the central question of our work: can we approach discontinuous problems in a way that preserves the low-variance benefits of pathwise gradients where possible, while using score-function estimators only where necessary?

Our goal is to develop a principled framework for reinforcement learning in environments with discontinuities, where standard pathwise gradient methods struggle. We introduce a modeling approach that explicitly captures the structure of such problems, enabling the use of pathwise gradients in regions where smoothness is available. Building on this framework, we propose methods that combine differentiable simulation with model-free gradient estimators, allowing us to leverage the strengths of both in a unified and theoretically grounded manner.

2.1. Problem types we aim to model

We present several examples of the types of discontinuities we aim to model:

- **Action set is a disconnected set (some regions are not single points).**
 - A supplier might only accept orders above a certain threshold.
 - An investment vehicle may require a minimum investment amount.
 - Loans may be offered in brackets; for example, small loans (1,000–100,000) are handled by “retail banking,” while large loans (500,000+) are handled by “wholesale banking.”
- **Cost is discontinuous with respect to actions.**
 - Vendors managing many products may ship all items together in a single truck or container.
 - A supplier may need to send an additional truck once certain quantity thresholds are reached.
 - A retailer might offer bulk discounts (or free shipping) when certain thresholds are met.
- **Transition is discontinuous with respect to actions.**
 - The lead time offered by a supplier may vary based on the quantity ordered.
 - If monetary resources are part of the state and we aim to maximize value, bulk discounts can induce discontinuities.

Brief overview of related work

Many real-world decision-making problems involve a *hybrid structure*, where part of the action is discrete (e.g., categorical modes) and part is continuous (e.g., intensities conditioned on the chosen mode). This setting arises in domains such as robotics (Xiong et al. 2018), supply chain management (Vanvuchelen et al. 2023), and smart energy systems (Huang et al. 2022). In such environments, agents must reason over both discrete mode switches and continuous control decisions, often in the presence of *discontinuous cost or transition dynamics*.

A promising strategy in these settings is to *combine score-function (SF) and pathwise (PW) gradient estimators*, leveraging their complementary strengths. SF estimators (e.g., REINFORCE (Williams 1992)) are applicable in non-differentiable settings and do not require knowledge of the problem’s dynamics, but often suffer from high variance. PW estimators, by contrast, yield lower-variance updates by directly differentiating through the environment, but require access to a differentiable model. The *Stochastic Computation Graph (SCG)* framework of Schulman et al. (2015a) formalizes how these two estimators can be composed depending on whether nodes are stochastic or deterministic. Several methods build on this foundation. Levy and Ermon (2018) proposed a hybrid policy gradient estimator by relaxing deterministic discrete-action policies into smooth stochastic ones, combining SF and PW updates. Rolfe (2016) introduced a similar mechanism in variational autoencoders by augmenting discrete latent variables with continuous surrogates to enable backpropagation. In reinforcement learning, Li et al. (2021) leveraged a conditional VAE to encode joint discrete-continuous actions into a latent space and optimized the policy via hybrid gradients. While these works demonstrate the feasibility of combining SF and PW estimators, they operate in *fully model-free or latent-variable settings*. In contrast, our work introduces a *structured modeling framework for piecewise-smooth MDPs (PS-MDPs)*, enabling principled gradient decomposition aligned with the known discontinuities in the environment, and exploiting pathwise gradients wherever smoothness is present.

Another line of work seeks to enable PW optimization by approximating discrete choices with *differentiable surrogates*. The *Gumbel-Softmax trick* (Jang et al. 2016, Maddison et al. 2016) provides a continuous relaxation of categorical distributions, enabling gradients to flow through sampled discrete variables. This is often combined with the *straight-through (ST) estimator* (Bengio et al. 2013), which samples a hard discrete action in the forward pass but applies gradients as if a soft relaxation were used. While these methods introduce bias, they reduce variance and have shown strong empirical performance in discrete-control settings. Our method incorporates these ideas but applies them in a *structure-aware manner*: we use ST estimators exclusively for discrete components, while applying unbiased PW estimators in smooth regions of the action space.

A third class of approaches tackles hybrid action problems using *fully model-free RL*. Xiong et al. (2018) introduced Parameterized DQN, which associates a continuous parameter with each discrete action and learns Q-values over the joint space. Huang et al. (2022) extended this idea to an actor-critic framework, training a DDPG agent for continuous control and a DQN agent for discrete selection, evaluating all combinations via a shared critic. Li et al. (2021) again used a VAE to represent discrete-continuous hybrid actions in a latent space, optimizing the policy using standard RL algorithms. In supply chain applications, Vanvuchelen et al. (2023) applied PPO to large-scale inventory control problems with hybrid decisions, relying on discretization and

heuristic mappings to reduce complexity. While these methods often perform well empirically, they treat the environment as a black box and do not exploit *pathwise gradients* available in smooth substructures. In contrast, our method selectively uses *differentiable simulation when available*, enabling stable and sample-efficient learning by combining low-variance PW gradients in smooth regions with unbiased SF estimators around discontinuities.

2.2. Main contributions

Our main contributions are as follows:

1. **Structured modeling framework:** We propose a factored modeling approach for reinforcement learning in piecewise-smooth MDPs, isolating smooth and discontinuous components. This facilitates stable learning in environments with large “jumps” in costs or transitions, among other nonsmooth behaviors.
2. **Hybrid policy optimization framework:** We leverage the Stochastic Computation Graph (SCG) formalism (Schulman et al. 2015a) to derive gradient estimators for factored policies with both deterministic and stochastic components. While similar hybrid estimators have been proposed (*e.g.*, Levy and Ermon (2018)), our contribution lies in *integrating these estimators into a structured learning framework* that explicitly exploits known smoothness in the environment.
3. **Hybrid-PPO and Straight-through HDPO:** We instantiate our framework in two algorithms. Hybrid-PPO blends Proximal Policy Optimization with pathwise gradients in smooth regions, while Straight-through HDPO extends HDPO using straight-through estimators.
4. **Empirical evaluation:** We demonstrate that our approach outperforms baseline methods by exploiting smoothness where available and combining gradients effectively. Our factored models improve gradient flow and stability, especially in problems with discontinuous costs and hybrid actions.

3. Problem Description

We begin by presenting a definition of Markov Decision Process (MDP).

3.1. Markov decision processes (MDPs)

In each period t , a decision-maker (DM) observes a state S_t , and chooses an action a_t from a feasible set $\mathcal{A}(S_t)$. In a T -period problem, a policy π is a sequence of functions $\pi = (\pi_1, \dots, \pi_T)$ in which each π_t maps a state S_t to a feasible action $\pi_t(S_t) \in \mathcal{A}(S_t)$. Let Π denote the set of feasible policies and $J(\pi) := \mathbb{E}^\pi \left[\frac{1}{T} \sum_{t \in [T]} c(S_t, \pi_t(S_t), \xi_t) \mid S_1 \right]$ denote the expected average cost of policy $\pi \in \Pi$ from initial state S_1 . The objective of the decision-maker is to solve

$$\min_{\pi \in \Pi} J(\pi) \quad \text{subject to} \quad S_{t+1} = f(S_t, \pi_t(S_t), \xi_t) \quad \forall t \in [T], \quad (2)$$

with $[x] = \{1, \dots, x\}$, and where the system function f governs state transitions, the cost function c governs per-period costs, and $\xi = (\xi_1, \dots, \xi_T)$ is a sequence of exogenous random terms. Note that the exogenous random terms may be drawn from an arbitrary stochastic process. We denote such an MDP as a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$.

In most RL problems, the decision-maker (DM) knows very little about the environment. In such a setting, the DM is able to improve the policy in a trial-and-error fashion using trajectories of observed states, actions and rewards. Throughout this work, we assume two additional properties: i) transition function f and cost function c are known and ii) the exogenous random terms are observable and exogenous. RL problems of this sort have been termed as *hindsight RL* (Alvo et al. 2023, Xie et al. 2024) and *exogenous MDPs* (Sinclair et al. 2022, Madeka et al. 2022). Collectively, these assumptions imply that for a given realization $\bar{\xi} = (\bar{\xi}_1, \dots, \bar{\xi}_T)$ of the stochastic process ξ , one can compute the performance of any policy in a counterfactual manner. Therefore, one can accurately estimate the performance of any policy using simulated rollouts on an **offline** dataset $\{\bar{\xi}^h\}_{h \in H}$ of T -length observations $\bar{\xi}^h = (\bar{\xi}_1^h, \dots, \bar{\xi}_T^h)$.

3.2. Piecewise smooth Markov decision processes

A Piecewise smooth MDP (PS-MDP) with J pieces has some additional properties, as defined below:

DEFINITION 1. An MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$ is a PS-MDP with J pieces if the following conditions hold: i) the action space $\mathcal{A} \subset \mathbb{R}$ is independent of the state and can be represented as the union of a finite set of disjoint intervals $\{\mathcal{B}^j\}_{j \in [J]}$ and ii) the cost and transition functions are continuous and a.e. differentiable w.r.t. S_t and ξ_t for every a_t and, for every interval $\mathcal{B}^j$, the *restrictions* of the cost and transition functions $c|_{\mathcal{B}^j}$ and $f|_{\mathcal{B}^j}$ to this interval are continuous and a.e differentiable w.r.t. a_t for every S and ξ .

The first condition above allows $\mathcal{A}$, *e.g.*, to be a disconnected set of intervals. Moreover, note that the second condition above allows, for example, large discontinuities in costs. Indeed, we only require that $c|_{\mathcal{B}^j}$ and $f|_{\mathcal{B}^j}$ are smooth within each interval, while we do not impose any conditions at the limit points of each of them.

3.3. Modeling approach

Our goal throughout this work is to exploit the smoothness properties within each interval $\mathcal{B}^j$. We therefore propose a modeling framework that allows to isolate the discontinuous and smooth components of the problem. This will allow us to propose algorithms that leverage smoothness, while appropriately addressing the discontinuities present.

For a given integer J , consider *factored action spaces* $\mathcal{B}^j$, and define *factored transition functions* $f^j : \mathcal{S} \times \mathcal{B}^j \times \Xi \rightarrow \mathcal{S}$ and *factored cost functions* $c^j : \mathcal{S} \times \mathcal{B}^j \times \Xi \rightarrow \mathbb{R}$ for each $j \in [J]$.

At each time step t , introduce binary variables $x_t^j \in \{0, 1\}$ and continuous variables $b_t^j \in \mathcal{B}^j$. The variable x_t^j indicates whether the j -th mode is selected at time t , and b_t^j specifies the action value within interval $\mathcal{B}^j$ if it is selected. Define the one-hot action selector set:

$$\mathcal{X}^J = \left\{ x \in \{0, 1\}^J \mid \sum_{j=1}^J x^j = 1 \right\}, \quad (3)$$

and the factored action space as:

$$\mathcal{A}^F = \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j. \quad (4)$$

Let $b = (b^1, \dots, b^J) \in \prod_{j=1}^J \mathcal{B}^j$ and define the vector-valued transition and cost outputs:

$$F(s, b, \xi) = (f^1(s, b^1, \xi), \dots, f^J(s, b^J, \xi)), \quad (5)$$

$$C(s, b, \xi) = (c^1(s, b^1, \xi), \dots, c^J(s, b^J, \xi)). \quad (6)$$

Define the *inner product* $\langle \cdot, \cdot \rangle : \mathbb{R}^J \times \mathbb{R}^J \rightarrow \mathbb{R}$ as:

$$\langle x, b \rangle := x^\top b. \quad (7)$$

Given that for $x \in \mathcal{X}^J$, exactly one of its values is 1 and all the rest are 0, this operation essentially selects the component of b corresponding to the active index in x .

3.4. Factored MDPs and Factored Representations

We begin by introducing factored MDPs, which provide a structure that allows us to isolate smooth components from discontinuous ones.

DEFINITION 2. An MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$ is a factored MDP of degree J if there exists $\{\mathcal{B}^j\}_{j \in [J]}$, $\{c^j\}_{j \in [J]}$ and $\{f^j\}_{j \in [J]}$ such that $\mathcal{A} = \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j$, $c(s, (x, b), \xi) = \langle x, C(s, b, \xi) \rangle$ and $f(s, (x, b), \xi) = \langle x, F(s, b, \xi) \rangle$ for all $s \in \mathcal{S}$, $(x, b) \in \mathcal{A}$, and $\xi \in \Xi$.

Additionally, if each $\mathcal{B}^j$ is connected and the functions c^j and f^j are continuous and a.e. differentiable with respect to $a \in \mathcal{B}^j$ for every $s \in \mathcal{S}$ and $\xi \in \Xi$, then we say that $\mathcal{M}$ is a smooth factored MDP of degree J .

For simplicity, we will overload notation and represent a factored MDP via the tuple $(\mathcal{S}, \{\mathcal{B}^j\}_{j \in [J]}, \{c^j\}_{j \in [J]}, \{f^j\}_{j \in [J]}, \Xi)$ throughout the remainder of this paper, which allows us to highlight the definition of the MDP via its factored components. We define the policy space of a factored MDP of degree J as $\Pi^J = \left\{ \pi = (\pi_1, \dots, \pi_T) \mid \pi_t : \mathcal{S} \rightarrow \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j \right\}$. We represent each mapping as a tuple $\pi_t = (\pi_t^{\mathcal{X}}, \pi_t^{\mathcal{B}})$, where $\pi_t^{\mathcal{X}} : \mathcal{S} \rightarrow \mathcal{X}^J$ defines the discrete mode and $\pi_t^{\mathcal{B}} : \mathcal{S} \rightarrow \prod_{j=1}^J \mathcal{B}^j$ outputs the continuous component for every mode.

We now define the notion of a factored representation, which allows us to express standard MDPs in terms of factored MDPs.

DEFINITION 3. Let $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$ be an MDP where $\mathcal{A} \subset \mathbb{R}$. We say that $\tilde{\mathcal{M}} = (\mathcal{S}, \{\mathcal{B}^j\}_{j \in [J]}, \{c^j\}_{j \in [J]}, \{f^j\}_{j \in [J]}, \Xi)$ is a factored representation of $\mathcal{M}$ of degree J if $\tilde{\mathcal{M}}$ is a factored MDP of degree J , $\mathcal{A} = \cup_{j \in [J]} \mathcal{B}^j$ and

$$c(s, \langle x, b \rangle, \xi) = \langle x, C(s, b, \xi) \rangle \quad (8)$$

$$f(s, \langle x, b \rangle, \xi) = \langle x, F(s, b, \xi) \rangle \quad (9)$$

for all $s \in \mathcal{S}$, $(x, b) \in \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j$, and $\xi \in \Xi$.

Additionally, we say $\tilde{\mathcal{M}}$ is a smooth factored representation of $\mathcal{M}$ of degree J if $\tilde{\mathcal{M}}$ is a smooth factored MDP.

In the previous definition, by virtue of $\mathcal{A} = \cup_{j \in [J]} \mathcal{B}^j$, we have that $\langle x, b \rangle = a \in \mathcal{B}^j \subseteq \mathcal{A}$, with j such that $x^j = 1$. Moreover, for every $a \in \mathcal{A}$, we can find $(x, b) \in \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j$ such that $\langle x, b \rangle = a$ by setting $b^j = a$ and taking x such that $x^j = 1$ if $a \in \mathcal{B}^j$ and $x^j = 0$ otherwise.

PROPOSITION 1. Consider the policy spaces Π and $\tilde{\Pi}$ of MDP $\mathcal{M}$ and a factored representation $\tilde{\mathcal{M}}$, respectively. For every $\pi \in \Pi$ there exist $\tilde{\pi} \in \tilde{\Pi}$ such that for every state $s \in \mathcal{S}$ and random term ξ the following holds:

$$c(s, \pi(s), \xi) = \langle \tilde{\pi}^{\mathcal{X}}(s), C(s, \tilde{\pi}^{\mathcal{B}}(s), \xi) \rangle \quad (10)$$

$$f(s, \pi(s), \xi) = \langle \tilde{\pi}^{\mathcal{X}}(s), F(s, \tilde{\pi}^{\mathcal{B}}(s), \xi) \rangle. \quad (11)$$

REMARK 1. The previous theorem essentially states that for every $\pi \in \Pi$, there is a policy $\tilde{\pi} \in \tilde{\Pi}$ such that the trajectories of states, actions (as defined in the their respective MDPs) and rewards are “equivalent” (in the sense that they describe the same quantities using a different representation) for every realization ξ of the stochastic process. If we have a policy $\tilde{\pi}$ in $\tilde{\mathcal{M}}$, we can construct such a policy by setting $\pi(s) = \langle \tilde{\pi}^{\mathcal{X}}(s), \tilde{\pi}^{\mathcal{B}}(s) \rangle$ for every s .

COROLLARY 1. For every policy $\pi \in \Pi$ in the original MDP, there exists a policy $\tilde{\pi} \in \Pi^J$ in the factored MDP such that $J(\pi) = J(\tilde{\pi})$. As a consequence, $\min_{\pi \in \Pi} J(\pi) = \min_{\tilde{\pi} \in \Pi^J} J(\tilde{\pi})$.

The previous result is not surprising, given that a factored MDP can be thought of a reformulation of an MDP. We state a more useful result below, alongside a constructive proof for illustrative purposes.

PROPOSITION 2. Every PS-MDP with J pieces has a smooth factored representation of degree J .

Proof Consider a PS-MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, c, f, \Xi)$ with J pieces. By definition, there exists a disjoint set of intervals $\{\mathcal{B}^j\}_{j \in [J]}$ such that $\mathcal{A} = \cup_{j \in [J]} \mathcal{B}^j$ and the restrictions of c and f to each $\mathcal{B}^j$, denoted

by $c|_{\mathcal{B}^j}$ and $f|_{\mathcal{B}^j}$, are continuous and a.e. differentiable with respect to b^j for each $s \in \mathcal{S}, \xi \in \Xi$. Consider the factored MDP defined by factored action spaces $\{\mathcal{B}^j\}_{j \in [J]}$, transition functions $f^j = f|_{\mathcal{B}^j}$ and cost functions $c^j = c|_{\mathcal{B}^j}$. Clearly, for every $s \in \mathcal{S}, (x, b) \in \mathcal{X}^J \times \prod_{j=1}^J \mathcal{B}^j$, and $\xi \in \Xi$ we have that $\langle x, C(s, b, \xi) \rangle = \sum_{j \in [J]} x^j \cdot C(s, b, \xi) = c(s, \langle x, b \rangle, \xi)$, with $\langle x, b \rangle = a$ for some $a \in \mathcal{A}$ (specifically, $a \in \mathcal{B}^j \subseteq \mathcal{A}$, with j such that $x^j = 1$).

3.5. Example: inventory control with fixed costs and varying lead times

We present a short example to show how to represent a PS-MDP with J pieces as a factored MDP of degree J .

EXAMPLE 1. Consider the setting of a retailer that sells a unique product across T periods. The retailer pays a linear underage cost p for unmet demand, and a linear holding cost h for carrying inventory at the end of each period. The retailer's supplier charges her fixed costs depending on the order size, as she needs to consolidate the orders into pallets: 0 for orders between 0 and 5 (as a "promotion", the first pallet is free), 10 for orders between 5 and 10, and 20 for orders between 10 and 20. Procurement costs per unit are negligible. Similarly, the supplier's lead time varies depending on the order size, given that she might need to buy additional input materials: orders between 0 and 7 can be delivered in 2 weeks, while orders between 7 and 20 in 3 weeks. They agree that all units in an order will be delivered simultaneously.

PS-MDP Representation. We can model this problem as a PS-MDP with 4 pieces as follows. The system state at time t is $S_t = (I_t, Q_t)$, where $I_t \in \mathbb{R}$ is the inventory on-hand and $Q_t \in \mathbb{R}_+^2$ tracks outstanding orders up to one minus the maximum lead time of 3 periods. The action at time t is the order quantity $a_t \in \mathbb{R}_+$.

Inventory on hand evolves as $I_{t+1} = \max(\{I_t - \xi_t, 0\}) + Q_t[1]$, where $Q_t[i]$ is the i -th component of Q_t . The outstanding orders update as

$$Q_{t+1} = \begin{cases} (Q_t[2] + a_t, 0), & \text{if } 0 \leq a_t \leq 7, \\ (Q_t[2], a_t), & \text{if } 7 < a_t \leq 20. \end{cases}$$

Finally, per-period costs are computed as

$$c(S_t, a_t, \xi_t) = p(\xi_t - I_t)^+ + h(I_t - \xi_t)^+ + \text{FixedCost}(a_t),$$

with

$$\text{FixedCost}(a_t) = \begin{cases} 0, & \text{if } 0 \leq a_t \leq 5, \\ 10, & \text{if } 5 < a_t \leq 10, \\ 20, & \text{if } 10 < a_t \leq 20. \end{cases}$$

Factored MDP Representation. We define a factored representation of the MDP described above with degree $J = 4$. The factored MDP is given by the tuple

$$\tilde{\mathcal{M}} = (\mathcal{S}, \{\mathcal{B}^j\}_{j \in [4]}, \{c^j\}_{j \in [4]}, \{f^j\}_{j \in [4]}, \Xi).$$

Factored action spaces are defined by:

$$\mathcal{B}^1 = [0, 5], \quad \mathcal{B}^2 = (5, 7], \quad \mathcal{B}^3 = (7, 10], \quad \mathcal{B}^4 = (10, 20].$$

Factored transition functions (which return pairs of the form (I_{t+1}, Q_{t+1})) are defined as:

$$f^j(S_t, b^j, \xi_t) = \begin{cases} (\max\{I_t - \xi_t, 0\} + Q_t[1], (Q_t[2] + b^j, 0)), & \text{for } j = 1, 2, \\ (\max\{I_t - \xi_t, 0\} + Q_t[1], (Q_t[2], b^j)), & \text{for } j = 3, 4. \end{cases}$$

Finally, cost functions are given by:

$$c^j(S_t, b^j, \xi_t) = p(\xi_t - I_t)^+ + h(I_t - \xi_t)^+ + \text{FixedCost}^j(b^j),$$

with FixedCost defined above.

3.6. Overview of our proposed approaches

Here, we give a high-level overview of how we will exploit the results in this section to solve the problems at-hand. Our proposed approaches operate on smooth factored MDPs of degree J , as defined in Definition 2, adhering to the following footprint. We consider a PS-MDP with J pieces, denoted by $\mathcal{M}$. Invoking Proposition 2, we can find a factored representation of degree J , denoted by $\tilde{\mathcal{M}}$. We will apply some algorithm on $\tilde{\mathcal{M}}$ obtaining a policy $\tilde{\pi}$ as a result. We then construct a policy π for the MDP $\mathcal{M}$ as $\pi(s) = \langle \tilde{\pi}^{\mathcal{X}}(s), \tilde{\pi}^{\mathcal{B}}(s) \rangle$ for every s . By Corollary 1, the cost of these policies in their respective MDPs is the same. By the same Corollary, if $\tilde{\pi}$ is optimal for $\tilde{\mathcal{M}}$, then π is optimal for $\mathcal{M}$. This is, operating on the factored MDP does not imply a degradation in costs.

4. Policy approximation and gradient computation

Our aim is to solve problems of realistic complexity using policy-based methods. In this section, we explain how to obtain unbiased gradient estimators for different classes of parameterized policies. Our main result will be showing that by partitioning a policy into a component that outputs a distribution over discrete values and another component that gives continuous values in a deterministic manner, we can obtain an unbiased gradient estimate composed of a combination of score-function and pathwise gradient terms. This will be conducive to algorithms that blend popular policy-gradient algorithms (*e.g.*, REINFORCE and PPO) with HDPO.

4.1. Gradient estimation via stochastic computation graphs

We compute policy gradients using the Stochastic Computation Graph (SCG) framework of Schulman et al. (2015a), which provides a flexible and intuitive method for deriving unbiased gradient estimators in models with both stochastic and deterministic components. This is particularly useful when randomness arises simultaneously from exogenous noise and stochastic policy elements.

Each node in an SCG belongs to one of four categories:

1. *Input nodes* (ψ): deterministic parameters that initiate the computation.
2. *Stochastic nodes*: random variables sampled from distributions conditioned on their parents.
3. *Deterministic nodes*: nodes that output deterministic functions of their parent values.
4. *Cost nodes*: terminal scalar nodes interpreted as loss terms.

Let $\mathcal{N}_{\text{input}}$, $\mathcal{N}_{\text{stoch}}$, $\mathcal{N}_{\text{det}}$, and $\mathcal{N}_{\text{cost}}$ denote the respective node sets. A directed edge $v \rightarrow w$ indicates that v influences w ; we write $v \rightarrow_D w$ if that influence path is purely deterministic (*i.e.*, it traverses only deterministic nodes).

Forward computation. We consider a single forward pass determined by

- ψ : a vector of input parameters,
- $\xi = (\xi_1, \dots, \xi_T)$: a trace of exogenous random variables, and
- ζ : auxiliary noise introduced by the stochastic policy components.

For every node v the forward pass produces a value $\hat{v}(\psi, \xi, \zeta)$. For a node w we collect the *deterministic history*

$$\hat{h}_w(\psi, \xi, \zeta) := (\hat{v}(\psi, \xi, \zeta) \mid v \rightarrow_D w).$$

Hence a stochastic node is drawn according to $p(\hat{w} \mid \hat{h}_w(\psi, \xi, \zeta))$.

For each stochastic node w define the total downstream cost

$$\hat{Q}_w(\hat{h}_w(\psi, \xi, \zeta)) := \sum_{\substack{c \in \mathcal{N}_{\text{cost}} \\ w \rightarrow_D c}} \hat{c}(\hat{h}_c(\psi, \xi, \zeta)),$$

where $\hat{c}(\hat{h}_c(\psi, \xi, \zeta))$ is the realised value of cost node c viewed as a deterministic function of its own history $\hat{h}_c(\psi, \xi, \zeta)$.

ASSUMPTION 1 (SCG differentiability). *Let ψ be an input node. For every node w with $\psi \rightarrow_D w$:*

- *If $w \in \mathcal{N}_{\text{det}}$, the Jacobian $\frac{\partial \hat{w}(\hat{h}_w(\psi, \xi, \zeta))}{\partial \psi}$ exists for all (ψ, ξ, ζ) .*
- *If $w \in \mathcal{N}_{\text{stoch}}$, the derivative*

$$\frac{\partial}{\partial \psi} \log p(\hat{w} \mid \hat{h}_w(\psi, \xi, \zeta))$$

exists for every realisation of $\hat{h}_w$.

Gradient computation. SCG theory decomposes the influence of parameters ψ into:

- **Score-function terms** for each stochastic node w :

$$\nabla_{\psi} \log p(\hat{w} \mid \hat{h}_w(\psi, \xi, \zeta)) \hat{Q}_w(\hat{h}_w(\psi, \xi, \zeta)),$$

weighted by the total downstream cost $\hat{Q}_w$.

- **Pathwise terms** for each cost node c that is deterministically influenced by ψ :

$$\nabla_{\psi} \hat{c}(\hat{h}_c(\psi, \xi, \zeta)).$$

THEOREM 1. *If ψ satisfies Assumption 1, then*

$$\begin{aligned} \frac{\partial}{\partial \psi} \mathbb{E}_{\xi, \zeta} \left[\sum_{c \in \mathcal{N}_{cost}} \hat{c}(\hat{h}_c(\psi, \xi, \zeta)) \right] &= \mathbb{E}_{\xi, \zeta} \left[\sum_{\substack{w \in \mathcal{N}_{stoch} \\ \psi \rightarrow_D w}} \frac{\partial}{\partial \psi} \log p(\hat{w} \mid \hat{h}_w(\psi, \xi, \zeta)) \hat{Q}_w(\hat{h}_w(\psi, \xi, \zeta)) \right. \\ &\quad \left. + \sum_{\substack{c \in \mathcal{N}_{cost} \\ \psi \rightarrow_D c}} \frac{\partial}{\partial \psi} \hat{c}(\hat{h}_c(\psi, \xi, \zeta)) \right]. \end{aligned}$$

Each term is computable by automatic differentiation as long as the underlying deterministic operations are differentiable. The clean separation between score-function and pathwise contributions allows each component of a hybrid policy to be trained with the most suitable gradient estimator.

4.2. Policy approximation for factored MDPs

Here, we present our policy approximations for smooth factored MDPs of the form $\tilde{\mathcal{M}} = (\mathcal{S}, \{\mathcal{B}^j\}_{j \in [J]}, \{c^j\}_{j \in [J]}, \{f^j\}_{j \in [J]}, \Xi)$. We consider parameterized functions that will define stochastic policies over the factored action space. The stochastic or deterministic nature of the mappings to the mode will lead to different estimators for the gradient of the expected cost with respect to policy parameters. In turn, this will be conducive to different algorithms.

Consider parameter spaces $\Phi = \mathbb{R}^{d_\phi}, \Theta = \mathbb{R}^{d_\theta}$, for some $d_\phi, d_\theta \in \mathbb{Z}$, and parameterized *policy components* $\tilde{\pi}_\phi^{\mathcal{X}}, \tilde{\pi}_\theta^{\mathcal{B}}$, for each $\phi \in \Phi$, and $\theta \in \Theta$, respectively. We will typically search over classes of neural policies, with ϕ and θ defining the weights of the neural networks. We say that a component $\tilde{\pi} \in \{\tilde{\pi}_\phi^{\mathcal{X}}, \tilde{\pi}_\theta^{\mathcal{B}}\}$ is *stochastic* if its output is a probability distribution over some set, and say it is *deterministic* if it maps to a fixed value. This resembles the concept of stochastic and deterministic policies. Policy components $\tilde{\pi}_\phi^{\mathcal{X}}$ and $\tilde{\pi}_\theta^{\mathcal{B}}$ can be stochastic or deterministic mappings from $s \in \mathcal{S}$ to the selector set $\mathcal{X}^J$ and $\mathcal{B}$, respectively. We will later specify the nature of each mapping, which will be conducive to different gradient estimators.

For a parameter ψ and stochastic policy component $\tilde{\pi}_\psi$, we will denote by $\tilde{\pi}_\psi(\cdot | s)$ the probability distribution given by the mapping on state s . If the mapping is deterministic, we will denote by $\tilde{\pi}_\psi(s)$ the value of the deterministic function. Given that only one coordinate in $x \in \mathcal{X}^J$ is 1 and all others are 0, one could define a factored approach by considering mappings $\tilde{\pi}_\theta^{\mathcal{B}}$ which are conditioned on x , and output a single real value. However, our proposed framework will allow for a seamless integration with algorithms that consider relaxations of the selector set $\mathcal{X}^J$, so we opt for such a formulation.

We assume the following assumptions throughout, which will allow for obtaining unbiased gradients.

ASSUMPTION 2 (Regularity of Policy Components). Let $\tilde{\pi}_\psi$ denote a policy component (either $\tilde{\pi}_\phi^\mathcal{X}$ or $\tilde{\pi}_\theta^\mathcal{B}$).

1. **Stochastic case.** If $\tilde{\pi}_\psi : \mathcal{S} \rightarrow \Delta(\mathcal{Y})$, then for every $(s, y) \in \mathcal{S} \times \mathcal{Y}$ the map $\psi \mapsto \tilde{\pi}_\psi(y | s)$ is continuously differentiable, and the score

$$\nabla_\psi \log \tilde{\pi}_\psi(y | s)$$

is square-integrable so that differentiation and expectation can be interchanged.

2. **Deterministic case.** If $\tilde{\pi}_\psi : \mathcal{S} \rightarrow \mathcal{Y}$, then for every $s \in \mathcal{S}$ the map $\psi \mapsto \tilde{\pi}_\psi(s)$ is almost-everywhere differentiable and locally Lipschitz in ψ . (Measurability in s is assumed throughout.)

For any pair (ϕ, θ) , we define the expected cumulative cost under the policy $\tilde{\pi} = (\tilde{\pi}_\phi, \tilde{\pi}_\theta)$ by $J(\phi, \theta)$. Additionally, for $x \in \mathcal{X}^J$, we denote by $j^*(x)$ the index j such that $x^j = 1$. Therefore, $b^{j^*(x)}$ will correspond to the only value in b that influences transitions and costs for the given x . At a high level, the marginal distribution of the j -th component of $\tilde{\pi}_\theta(\cdot | s)$ can be interpreted as the distribution of the continuous value conditional on x . We define the state–action value function under policy $(\tilde{\pi}_\phi, \tilde{\pi}_\theta)$ at time t as

$$Q_t^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}(s, a) := \mathbb{E} \left[\frac{1}{T} \sum_{t'=t}^T c(S_{t'}, A_{t'}, \xi_{t'}) \mid S_t = s, A_t = a \right], \quad (12)$$

where $a = (x, b)$, and future actions are sampled according to $(\tilde{\pi}_\phi, \tilde{\pi}_\theta)$.

Letting S_t , X_t and B_t be random variables denoting the state, discrete action and continuous action observed at time t , we define the state–action occupancy measure induced by the policy $(\tilde{\pi}_\phi, \tilde{\pi}_\theta)$ as

$$d_\infty^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}(s, (x, b)) := \frac{1}{T} \sum_{t=1}^T \Pr(S_t = s, X_t = x, B_t = b \mid \tilde{\pi}_\phi, \tilde{\pi}_\theta).$$

This quantity represent the “average time” we spend in each particular (state, action) pair, and will allow us to define expectations in a compact way.

4.3. Gradient estimation for several policy classes

We now consider two specializations of the policy approximations defined in the previous section, which differ in the nature of the mappings defined by $\tilde{\pi}_\theta^\mathcal{B}$. We present, for each algorithm, an SCG that spans two decision periods. Noise nodes (*i.e.*, ξ and ζ) are included only when their realized values are actually required for computing the gradient. For instance, knowledge of the realized ξ is needed when computing pathwise gradients, as it is required to assess counterfactuals (*i.e.*, how costs change when a policy parameter is perturbed). On the other hand, while ζ influences the likelihood of sampled values at stochastic nodes, its realized value is not needed to compute either

pathwise or score-function terms. Consequently, SCGs that rely on pathwise gradients include ξ , whereas no diagram contains ζ .

We define the class of *fully stochastic* policies Π_1 as the set of parameterized policies such that $\tilde{\pi}_\phi$ is a stochastic mapping from state $s \in \mathcal{S}$ to a distribution $F_1 \in \Delta(\mathcal{X}^J)$ and $\tilde{\pi}_\theta$ is a stochastic mapping from state $s \in \mathcal{S}$ to a (product-form) distribution $F_2 \in \Delta(\mathcal{B})$. We show the computation graph associated to policies in this class in Figure 1a. Under this policy, the only deterministic nodes are either inputs or costs. This will lead to our gradient estimators be composed of score-function terms for every policy component node.

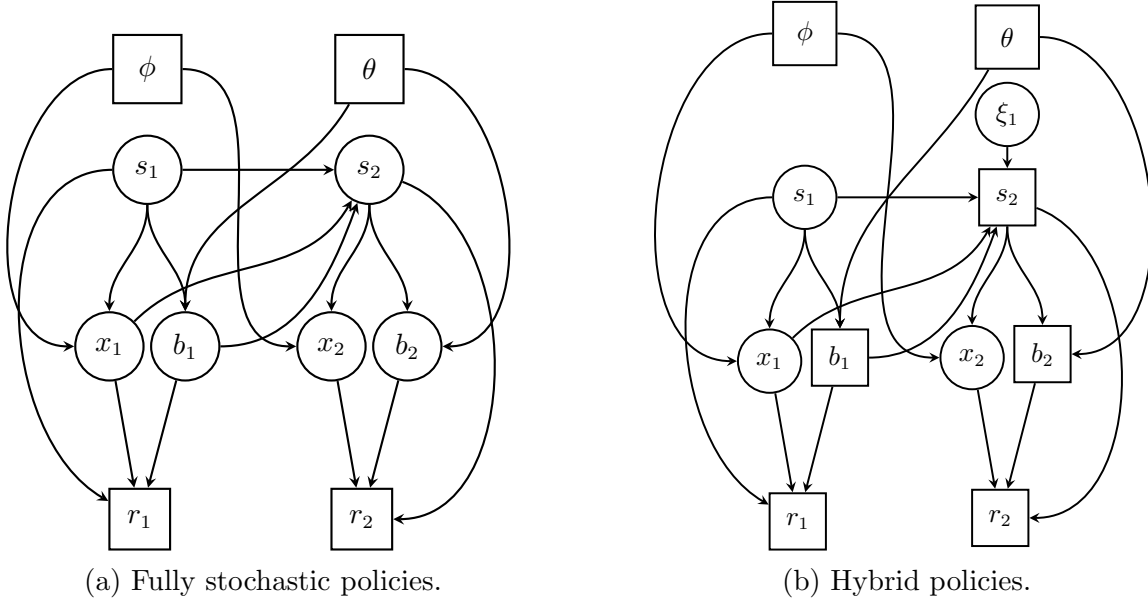


Figure 1 Comparison of computation graph structures.

THEOREM 2 (Unbiased gradient estimator for fully stochastic policies). *Under assumption 2, the following holds for every fully stochastic policy $\tilde{\pi} = (\tilde{\pi}_\phi, \tilde{\pi}_\theta) \in \Pi_1$:*

$$\mathbb{E}_{(s,a) \sim d_\infty^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}} \left[\nabla_{(\phi, \theta)} \left(\log \tilde{\pi}_\phi(x | s) + \log \tilde{\pi}_\theta(b^{j^*(x)} | s) \right) Q_t^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}(s, a) \right] = \nabla_{(\phi, \theta)} J(\phi, \theta). \quad (13)$$

Policies in this class do not exploit any knowledge about the model, and can only learn via sampled trajectories. This leads to a score-function gradient estimator such as the one in (13) to have a very large variance. It is noteworthy that, for a policy $\tilde{\pi} = (\tilde{\pi}_\phi^{\mathcal{X}^J}, \tilde{\pi}_\theta^{\mathcal{B}})$ in which both mappings are stochastic, we can directly apply several classical policy-gradient algorithms, such as REINFORCE (Williams 1992), Trust Region Policy Optimization (Schulman et al. 2015b) and Proximal Policy Optimization (Schulman et al. 2017).

If we possess knowledge about the transition and cost functions and assume that the stochastic process is exogenous and observable, we can obtain a gradient estimator which typically has a lower

variance. We define the class of *hybrid* policies Π_2 as the set of parameterized policies such that $\tilde{\pi}_\phi$ is a stochastic mapping from state $s \in \mathcal{S}$ to action component $x \in \mathcal{X}^J$ and $\tilde{\pi}_\theta$ is a deterministic mapping from state $s \in \mathcal{S}$ to $b \in \mathcal{B}$.

We show the computation graph associated to policies in this class in Figure 1b. There are 2 main differences with respect to the computation graph of fully stochastic policies. First, nodes b_1 and b_2 are now deterministic. Second, we split the computation of state s_2 into a stochastic computation in ξ_1 and a deterministic mapping s_2 of its inputs. This is sometimes referred to as the *reparameterization trick* (Kingma et al. 2013, Rezende et al. 2014). Now, θ deterministically influences rewards r_1 and r_2 . In particular, θ deterministically influences r_2 not only via its influence on b_2 on the path $\theta \rightarrow b_2$, but also through the path $\theta \rightarrow b_1 \rightarrow s_2 \rightarrow b_2$. Moreover, θ deterministically influences x_2 via the path $\theta \rightarrow b_1 \rightarrow s_2 \rightarrow x_2$.

THEOREM 3 (Unbiased gradient estimator for hybrid policies). *Under assumption 2, the following holds for every hybrid policy $\tilde{\pi} = (\tilde{\pi}_\phi, \tilde{\pi}_\theta) \in \Pi_2$:*

$$\mathbb{E}_{(s,a) \sim d_{\infty}^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}} \left[\nabla_{(\phi, \theta)} \log \tilde{\pi}_\phi(x | s) Q_t^{(\tilde{\pi}_\phi, \tilde{\pi}_\theta)}(s, a) + (\mathbf{0}_{d_\phi}, \nabla_\theta c(s, a)) \right] = \nabla_{(\phi, \theta)} J(\phi, \theta), \quad (14)$$

where $\mathbf{0}_{d_\phi}$ is a vector of zeroes of dimension d_ϕ .

In the previous theorem, note that the score-function term considers only the policy component $\tilde{\pi}_\phi$, while the gradient is taken with respect to (ϕ, θ) . This term reflects the direct influence of ϕ on $\tilde{\pi}_\phi$ and, also, the influence of θ via its influence on s_1 . The second term in the expectation reflects the influence of θ in the costs (via **all** deterministic paths connecting θ and the cost), while representing that ϕ does not deterministically influence costs.

REMARK 2 (REPARAMETERIZATION TRICK FOR STOCHASTIC $\tilde{\pi}_\theta$). One can alternatively obtain a gradient similar to that of (14) by applying the reparameterization trick to a stochastic $\tilde{\pi}_\theta$ under appropriate distributional assumptions (*e.g.*, when $\tilde{\pi}_\theta$ is Gaussian). This entails expressing samples of the continuous component as deterministic functions of their inputs and independent noise, analogous to the noise injection used in the computation of s_2 in Figure 1b. Experimental results in Section 6 suggest that deterministic mappings for $\tilde{\pi}_\theta$ already perform well, so we do not pursue this variant here.

4.4. Fully deterministic policies via straight-through gradients

We propose an additional approach, in which both discrete and continuous action components are deterministic, which only necessitates pathwise gradients. In principle, using pathwise gradients for discrete actions leads to uninformative gradients. To get an informative gradient signal, we utilize straight-through gradients, as proposed by Bengio et al. (2013), on the discrete actions. Our approach is as follows. We use a softmax as the last layer for the neural networks related to

the discrete actions. On the forward pass, we deterministically select the discrete action with the highest value. During the backwards pass, we compute gradients as if we had selected actions using a softmax.

We refer to these policies as *fully deterministic* ones, and define the class of such policies as Π_3 . For policies in this class, we consider a gradient estimator of the form $\mathbb{E}_{(s,a) \sim d_\infty^{(\bar{\pi}_\phi, \bar{\pi}_\theta)}} [\nabla_{(\phi, \theta)} c(s, a)]$. We emphasize that this gradient is likely biased (for discrete actions, the gradient is undefined whenever there are ties in the largest output, and zero elsewhere). However, such an estimator is expected to enjoy smaller variance than that of the hybrid policy (see Eq. (14)), given that we estimate the gradient of discrete components using pathwise gradients instead of score-function ones.

REMARK 3 (GUMBEL-SOFTMAX TRICK ON THE DISCRETE ACTION COMPONENT). One alternative to obtain pathwise gradients for the discrete actions is to use the Gumbel-softmax trick (Jang et al. 2016, Maddison et al. 2016). Under this approach, one typically injects Gumbel noise to the outputs of the last linear layer and divide by a value τ (the *temperature*). A softmax function is then applied to these outputs, leading to a continuous approximation of the discrete values. The temperature τ is typically set to large values initially, leading to a smooth but very biased approximation. As training progresses, the temperature can be *annealed* to smaller values, leading to a closer approximation. Moreover, one can combine this technique with straight-through gradients by simulating feasible discrete action using an “argmax” on the forward pass, but computing gradients as if a softmax was used in the backward pass. Numerical results suggest that using fully deterministic policies works better than employing the Gumbel-softmax trick, so we do not further analyze this variant.

5. Algorithms

In this section, we propose several algorithms built on top of the results and insights developed in Section 4. First, we give a quick overview of how to obtain feasible policies for our factored representations. For a PS-MDP with J pieces, consider a factored MDP representation of degree J , $\tilde{\mathcal{M}} = (\mathcal{S}, \{\mathcal{B}^j\}_{j \in [J]}, \{\mathcal{C}^j\}_{j \in [J]}, \{f^j\}_{j \in [J]}, \Xi)$. We assume that each $\mathcal{B}^j = [\underline{d}^j, \bar{d}^j]$ for some real values, that can possibly be $\pm\infty$. To ensure our policies output feasible values, we compute the continuous components by applying the following transformations to the corresponding output o^j :

$$b^j = \begin{cases} o^j, & \text{if } \underline{d}^j = -\infty \text{ and } \bar{d}^j = +\infty \\ \underline{d}^j + \text{sigmoid}(o^j)(\bar{d}^j - \underline{d}^j), & \text{if } \underline{d}^j, \bar{d}^j \text{ are finite} \\ \bar{d}^j - \text{softplus}(o^j), & \text{if } \underline{d}^j = -\infty \\ \underline{d}^j + \text{softplus}(o^j), & \text{if } \bar{d}^j = +\infty \end{cases} \quad (15)$$

We consider algorithms based on the unbiased gradient estimators for fully stochastic, hybrid and fully deterministic policies presented in Section 4. Additionally, we detail a benchmark which directly tries to solve the MDP by utilizing a single continuous action.

5.1. Building a surrogate function

Our goal is to obtain gradients that can be computed with modern automatic-differentiation libraries such as PYTORCH. To that end, we construct a *surrogate objective* whose ordinary gradient coincides in expectation with the true policy gradient for the three policy classes of Section 4. The construction below is a direct restatement of Corollary 1 in Schulman et al. (2015a) using our notation.

Surrogate loss. For parameters (ϕ, θ) , an initial state S_1 , and a realization of exogenous noise $\xi = (\xi_1, \dots, \xi_T)$, define

$$L(\phi, \theta; S_1, \xi, \zeta) := \sum_{w \in \mathcal{N}_{\text{stoch}}} \log p(\hat{w} \mid \hat{h}_w(\psi, \xi, \zeta)) \hat{Q}_w + \sum_{c \in \mathcal{N}_{\text{cost}}} c(\hat{h}_c(\psi, \xi, \zeta)),$$

where $\hat{w}$, $\hat{h}_w(\psi, \xi, \zeta)$, and $\hat{Q}_w$ are the sampled values produced in the forward pass described in Section 4.1.

COROLLARY 2 (Surrogate objective for hybrid policies). *Differentiating the surrogate yields an unbiased estimator of the true gradient:*

$$\mathbb{E}_{\xi, \zeta} [\nabla_{(\phi, \theta)} L(\phi, \theta; S_1, \xi, \zeta)] = \mathbb{E}_{(\tilde{\pi}_{\phi}^{\mathcal{X}}, \tilde{\pi}_{\theta}^{\mathcal{B}})} \left[\sum_{t=1}^T c(S_t, A_t, \xi_t) \right].$$

5.2. Proposed algorithms

Corollary 2 implies that one can construct algorithms combining pathwise and score-function gradients by evaluating $L(\phi, \theta; S_1, \xi, \zeta)$ on samples (ξ, ζ) . In particular, for a hindsight RL problem (*i.e.*, when the model is known and noise is observable), one can obtain unbiased gradient estimates by computing this objective over simulated rollouts using an **offline** dataset $\{\bar{\xi}^h\}_{h \in [H]}$ consisting of T -length demand traces $\bar{\xi}^h = (\bar{\xi}_1^h, \dots, \bar{\xi}_T^h)$. Gradients can then be computed using automatic differentiation tools of the practitioner’s choice.

We next spell out the concrete learning rules used in our experiments. Whenever a gradient term is of *score-function* type we apply “tricks” commonly used in PPO (learned critic for advantage estimates, reward normalisation, clipped probability ratios, generalized advantage estimation, among others). Whenever a term is of *pathwise* type we treat it with HDPO; the only extra heuristic we retain there is reward normalisation.

The resulting four algorithms are:

1. **Proximal Policy Optimization (PPO)** (Schulman et al. 2017). We run vanilla PPO on the factored MDP, using the fully-stochastic gradient in (13). The policy is fully stochastic and factored: a discrete mode is sampled first and the continuous component is sampled *conditionally* on that mode.

2. **Hybrid-PPO.** Starting from the hybrid gradient in (14), we *blend* PPO and HDPO: score-function terms receive the PPO treatments while pathwise terms are taken exactly as in HDPO.
3. **Straight-Through HDPO (ST-HDPO).** Here the policy is fully deterministic; we use HDPO together with straight-through gradients as described in Section 4.4.
4. **Continuous HDPO.** HDPO is applied directly on the original PS-MDP, approximating each discontinuity with a steep but continuous surrogate so that all gradients are pathwise.

6. Numerical experiments

We assessed the 4 benchmarks in a variant of the problem defined in Example 1. We consider a fixed lead time of 2 periods, fixed costs of 0 for no orders, and a large fixed cost of 64 for positive orders (in practice, we consider a small positive value and consider the fixed cost for orders above this value). We consider a holding cost of 1, underage cost of 9, a stationary Poisson demand with mean 10, and assume that unmet demand is backlogged. Under this conditions, an (s, S) policy is optimal. This policy orders whenever the inventory position (sum of inventory on hand and outstanding orders) is below or equal to s , and an order to raise the position to S is made. In this setting, we can compare to a certifiable optimum.

Table 3 shows the cost attained and optimality gap for each of the algorithms on their best run (we select on a held-out validation set and report performance on a clean test set). Given that these are preliminary experiments, the number of runs and hyperparameters considered for each algorithms are highly heterogeneous, so the conclusions drawn from them are only suggestive.

Table 3 Optimality gap of each algorithm (baseline cost $c^* = 37.06$)

Algorithm	Cost	Optimality gap (%)
PPO	37.33	0.73
Hybrid-PPO	37.13	0.19
ST-HDPO	37.09	0.08
Continuous HDPO	46.07	24.30

As reflected in the table, Continuous HDPO achieves results that are drastically worse than all other methods. Since Continuous HDPO is the only method that is agnostic to the piecewise-continuity structure of the problem, this reveals that there is a lot of value in defining a model (or policy) that reflects these underlying property. Moreover, these results show a significant improvement of Hybrid-PPO with respect to PPO. We additionally tested the two algorithms in the same setting, but assuming that unmet demand is lost. In this setting, Hybrid-PPO outperformed PPO by 0.29%. Collectively, these results suggests that there is significant value in exploiting continuity

and differentiability within each piece, as opposed to treating the problem as a “black-box”. We hypothesize that the gaps between Hybrid-PPO and PPO will become larger for high-dimensional continuous outputs, where model-free methods typically perform poorly.

Finally, we note that ST-HDPO performed slightly better than Hybrid-PPO, even though it operates using biased gradient estimates. We expect the benefits of ST-HDPO over Hybrid-PPO to become larger for an increased number of possible discrete values (or for high-dimensional problems), given the poor scaling of PPO and other model-free methods in this regime.

7. Conclusion and future steps

Early results suggest that our proposed algorithms are effective at exploiting the differentiable structure within each “piece” of the problem. However, their implementation—particularly that of Hybrid-PPO—posed significant engineering challenges. For example, the score-function and pathwise gradient components appear to operate at vastly different scales. Addressing this required, among other things, normalizing gradients separately for each term and using distinct learning rates. Developing a more principled method to balance these two components would significantly improve the practicality of the algorithm.

In addition, our numerical evaluation remains narrow in scope. Testing these methods on problems of a different nature would help strengthen our conclusions. In particular, demonstrating the benefits of the hybrid approach on classical benchmark domains beyond operations—such as tasks from the MuJoCo suite—would make the value of our contribution more evident.

References

- Alvo M, Russo D, Kanoria Y (2023) Neural inventory control in networks via hindsight differentiable policy optimization. *arXiv preprint arXiv:2306.11246* .
- Bengio Y, Léonard N, Courville A (2013) Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* .
- Che E, Dong J, Namkoong H (2024) Differentiable discrete event simulation for queuing network control. *arXiv preprint arXiv:2409.03740* .
- Hu Y, Anderson L, Li TM, Sun Q, Carr N, Ragan-Kelley J, Durand F (2019) DiffTaichi: Differentiable programming for physical simulation. *arXiv preprint arXiv:1910.00935* .
- Huang C, Zhang H, Wang L, Luo X, Song Y (2022) Mixed deep reinforcement learning considering discrete-continuous hybrid action space for smart home energy management. *Journal of Modern Power Systems and Clean Energy* 10(3):743–754.
- Jang E, Gu S, Poole B (2016) Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* .
- Kingma DP, Welling M, et al. (2013) Auto-encoding variational bayes.
- Levine S, Finn C, Darrell T, Abbeel P (2016) End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research* 17(1):1334–1373.
- Levy D, Ermon S (2018) Deterministic policy optimization by combining pathwise and score function estimators for discrete action spaces. *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32.
- Li B, Tang H, Zheng Y, Hao J, Li P, Wang Z, Meng Z, Wang L (2021) Hyar: Addressing discrete-continuous action reinforcement learning via hybrid action representation. *arXiv preprint arXiv:2109.05490* .
- Maddison CJ, Mnih A, Teh YW (2016) The concrete distribution: A continuous relaxation of discrete random variables. *arXiv preprint arXiv:1611.00712* .
- Madeka D, Torkkola K, Eisenach C, Foster D, Luo A (2022) Deep inventory management. *arXiv preprint arXiv:2210.03137* .
- Mohamed S, Rosca M, Figurnov M, Mnih A (2020) Monte carlo gradient estimation in machine learning. *The Journal of Machine Learning Research* 21(1):5183–5244.
- Rezende DJ, Mohamed S, Wierstra D (2014) Stochastic backpropagation and approximate inference in deep generative models. *International conference on machine learning*, 1278–1286 (PMLR).
- Rolfe JT (2016) Discrete variational autoencoders. *arXiv preprint arXiv:1609.02200* .
- Schulman J, Heess N, Weber T, Abbeel P (2015a) Gradient estimation using stochastic computation graphs. *Advances in neural information processing systems* 28.

-
- Schulman J, Levine S, Abbeel P, Jordan M, Moritz P (2015b) Trust region policy optimization. *International conference on machine learning*, 1889–1897 (PMLR).
- Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* .
- Sinclair SR, Frujeri F, Cheng CA, Swaminathan A (2022) Hindsight learning for mdps with exogenous inputs. *arXiv preprint arXiv:2207.06272* .
- Vanvuchelen N, De Moor B, Boute R (2023) The use of continuous action representations to scale deep reinforcement learning for inventory control .
- Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Reinforcement learning* 5–32.
- Xie Y, Ma W, Xin L (2024) Vc theory for inventory policies. *arXiv preprint arXiv:2404.11509* .
- Xiong J, Wang Q, Yang Z, Sun P, Han L, Zheng Y, Fu H, Zhang T, Liu J, Liu H (2018) Parametrized deep q-networks learning: Reinforcement learning with discrete-continuous hybrid action space. *arXiv preprint arXiv:1810.06394* .